

Masters Thesis 2023

Study on designing artificial Pokémon masters

A Thesis Submitted to the Department of Computer Science and
Communications Engineering, the Graduate School of Fundamental Science
and Engineering of Waseda University in Partial Fulfillment of the
Requirements for the Degree of Master of Engineering

submitted on 2023.07.24

Waseda University
School of Fundamental Science and Engineering
Department of Computer Science

5121FG30-5 Chen Kaishang

Supervisor Edgar Simo-Serra
Area of Study Computer Graphics

Abstract

Pokémon is one of the most celebrated video game franchises in history and competitive Pokemon battling has earned increasing popularity as an esports over the last decade. Competitive Pokemon battle poses a significant challenge to developing AI agents due to partial observability, long-term causal relationship between states and actions, an enormous knowledge base as well as the raw complexity of the game system itself, which are also representative of issues found in many real-world tasks. Self-play reinforcement learning has shown promising results in other well-studied games such as Go and Shogi, but we struggled to achieve good performance with existing self-play methods in Pokemon, as the agents often struggle to get out of local optimums, possibly due to the partial observability and random nature of the game. To overcome this, we introduce a novel multi-agent competition based framework to train reinforcement learning agents to play pokemon. Experiments showed that agents trained with our method outperformed those trained with self-play, even without reward shaping.

Acknowledgements

This thesis would not have been possible without the generous help from my instructor Edgar Simo-Serra and support from PhD senior Kotaro Kikuchi. They provided crucial technical support during the course of this research and has been incredible friends. I am also deeply grateful to all other friends that stayed together with me and provided mental supports during this long-lasting pandemic.

Contents

1	Introduction	6
1.1	Scope and aim of this thesis	7
2	Background	9
2.1	Games	9
2.1.1	Two-player zero-sum extensive games	10
2.2	Regret Minimization Methods	16
2.2.1	Regret and Regret Matching	17
2.2.2	CounterFactual Regret(CFR)	21
2.2.3	Monte Carlo CounterFactual Regret minimization(MC-CFR)	25
2.3	reinforcement learning and deep RL	27
2.3.1	Reward Shaping	29
2.3.2	Value functions, policies and models	30
2.4	Neural Networks	34
2.5	Loss Functions	36
2.6	Stochastic Gradient Descent and Backpropagation	39
2.7	Policy Gradient Methods	41
2.7.1	REINFORCE	43
2.7.2	Baselines and Weighting Factors	44
2.7.3	Proximal Policy Optimization(PPO)	45
2.8	Self-Play	47
2.9	Pokemon	48
2.9.1	Basic Damage Calculation	48
2.9.2	Type Effectiveness	49
3	Related Works	51
3.1	AlphaGo	51
3.2	Population-based Training	52
3.3	Self-play RL Pokemon Agent: Metagrok	52

<i>CONTENTS</i>	2
4 Methodology	53
4.1 Game engine and learning environment	53
4.2 reinforcement learning baseline	55
4.2.1 reward shaping	56
4.3 Self-play RL	56
4.4 Population-based RL	57
5 Experiments and Results	59
5.1 Evaluation Opponents	59
5.2 Experiment Results	60
6 Conclusions and future works	62
6.1 Expanding observation space	62
6.2 Importance of search	63
APPENDIX	71

List of Figures

2.1	Simple example of a extensive game with prefect information. There are 2 players $P = 1, 2$ and each player can take action to go either up (U) or down (D). After both players have taken actions, the game reaches a terminal state and each player are given payoff. Notice this is not zero-sum, as the payoff does not add up to zero.	11
2.2	Simple example of a extensive game with prefect information. There are 2 players $P = 1, 2$ and each player can take action to go either up (U) or down (D). After both players have taken actions, the game reaches a terminal state and each player are given payoff. Notice this is not zero-sum, as the payoff does not add up to zero.	19
2.3	Example of applying self-play regret matching on rock-paper-scissor. After training for 25,000 iterations, action probabilities for all three actions start to converge to Nash equilibrium.	22
2.4	The agent-environment interaction is a markov decision process. Figure taken from [25]	28
2.5	A generic feedforward Neural Network. Figure taken from [25]. Each circle represents a perceptron that calculated a weighted sum with bias for all its input and then activate the result with some activation function before sending it as output. Arrows represent inputs and outputs. For one perceptron, inward-pointing arrows are inputs coming from previous layers and all outward-pointing arrows represent the same output value of that perceptron.	33
2.6	Commonly used activation functions in neural networks	35

- 2.7 Type chart of Pokémon (Figure by Aussie Evil(CC0)). How to read this figure: to find the damage multiplier of Fire type on Grass type, first find Fire on the second row from the verticle Attacker column, then from that row find Grass type on the forth column, which reads 2, therefore Fire type attacks have a 2X multiplier on Grass type Pokemons. 50

List of Tables

2.1	Evolution of regret values over the first three iterations	21
2.2	Evolution of strategy over the first three iterations of regret matching	21
5.1	Evaluation win rates against heuristic bots	60
6.1	Machine specs	70
6.2	Mathematical notations used in game definition	73
6.3	Mathematical notations used extensive games and regret min- imization	74
6.4	Mathematical notations used in reinforcement learning	75

Chapter 1

Introduction

Since the beginning of the field, games, from classic board games to modern video games, has inspired and incubated a forest of innovations and breakthroughs in AI. Some might say that games to Artificial Intelligence (AI) researchers are much like laboratory mice to biomedical researchers.

In the 1950s, Authur L. Samuel developed a Checkers-playing program that utilized algorithms later known as minimax search, "rote learning" and self-play[1]. This work inspired many others to ultimately found the field of AI and popularized the term "machine learning"[2]. Later in the 1990s, TD-Gammon managed to teach itself to play Backgammon at human master level[3] through self-play and IBM's historic DeepBlue beat human world champion at chess taking advantage of features constructed on expert knowledge[4].

Since the deep learning revolution from 2012[5], game AIs have also adapted to make use of the representation power of deep neural networks and start beating humans in even more sophisticated games. Deepmind's AlphaGo beat the human world champion in the game of Go in 2016 by leveraging large amount of human expert data with deep learning followed

by self-play reinforcement learning and search[6]. Later in 2017 AlphaZero beat AlphaGo 100-0 using self-play reinforcement learning and search only[7].

With deep learning, we also start to see the game-inspired methodologies to start being applied to solve problems outside of games with real-world stakes. A variant of AlphaZero called MuZero[8] were later employed to compose video compression algorithm that achieves higher compress rate than the industry standard while maintaining similar video quality[9]. Another variant of AlphaZero managed to discover a novel algorithm to perform matrix multiplication faster than Strassen’s two-level algorithm, making progress in the field for the first time in more than 50 years[10]. Techniques that powered AlphaZero such as Monte-Carlo tree search were also applied to build self-driving automobile systems to park themselves[11].

1.1 Scope and aim of this thesis

Inspired by the long fruitful history of game AI research, this thesis aims to tackle a game that has yet to be solved by AI systems: Pokémon. Pokémon, in a nutshell, is a game like the mixture of Poker and chess, with added complexity and stochasticity. It resembles chess in the sense that positioning oneself to be in advantageous is crucial in winning the game, i.e. how a player sets himself up in the early turns of a game has significant effect on the probability of that player winning. It is comparable to Poker because of its partial observability, which means key information such as the opponent’s card is invisible to the player, therefore it is important for learning agents to learn either implicitly or explicitly the probability distribution of hidden cards to make optimal decisions. And unlike chess and Poker, Pokémon has rich components of randomness, meaning the same action might have

dramatically different effects on the game depending on a dice roll, which, one might argue captures the fundamental probabilistic nature of our ground reality.

In this thesis we first survey methods that have been historically successful in solving classic board games such as chess, go, and poker. The researches on perfect information game such as chess and go and the researches on imperfect information game such as poker turn out to be two distinct families of studies that start from different roots and later converge into one extensive area of study. Both branches involve game theory as foundations, but the Poker branch for the past 20 years has been developing on an idea named "regret minimization". The chess and go branch on the other hand builds upon the idea of tree search and reinforcement learning. We detail the evolution of both branches of study in chapter 2: backgrounds.

In chapter 3, we will introduce some recent works that combine regret minimization and reinforcement learning with modern deep neural networks. In chapter 4 and 5, we conduct a few experiments to test these algorithms' effectiveness on the game of Pokémon and discuss the results. And finally in chapter 6, we will discuss future potential future directions of this research.

Chapter 2

Background

This chapter lays out the core ideas that are relevant to this thesis, such as definitions of extensive game, basic ideas of machine learning and reinforcement learning as the like. Details such as network architecture, hyperparameter settings, normalization and regularization during learning will be discussed in later chapters. Explanation of the game of Pokemon will also be provided at the end of this chapter.

2.1 Games

A game is a structured sequence of interactions between an autonomous being and the environment or other autonomous beings. A game is usually created, or emerges out of nature to serve the mammalian instinct of play. Lion cubs play wrestling against each other to learn the necessary muscle movements and tactics later useful in hunting. Many dogs are born with the instinct to play throw and catch with humans. And human beings play all kinds of games with and against each other, mostly for entertainment, and some as professions.

Games in human cultures takes wide-ranging forms, from the most simplest rock-paper-scissor to modern photo-realistic video games such as Grand Theft Auto 5. Regardless of the form, all games abstracts structure of dynamics as well as objectives from reality, such as moving from point A to point B while collecting objects of value along the way (Super Mario Series), simply exploring the terrain (Minecraft), and most importantly, achieving victory of some sort.

Most objectives existing in games have real world counterparts, and with much of the complexity and unpredictability abstracted away, games are perfect test grounds for AI research. In this work, we focus on the game of Pokemon, a two-player zero-sum extensive game. Another famous game of this category is all forms of Poker. In the following subsections, we will provide formal definitions of such games.

2.1.1 Two-player zero-sum extensive games

A two-player zero-sum game, as the name suggests, is a game played between two players with the total reward sums up to zero. Formally, let $P = \{1, 2\}$ be the set of players of the game and u_p be the payoff (reward) function of player p , then:

$$u_1 + u_2 = 0$$

Examples of such games include rock-paper-scissor, chess, poker and Pokemon. When one player wins ($u_1 = +1$), the other one losses ($u_2 = -1$).

A extensive-form game is a game that is typically iterated over turns and the next state is dependent on players' $P = p_1, p_2$ action input on the current turn. Such game can be intuitively represented with a tree data structure,

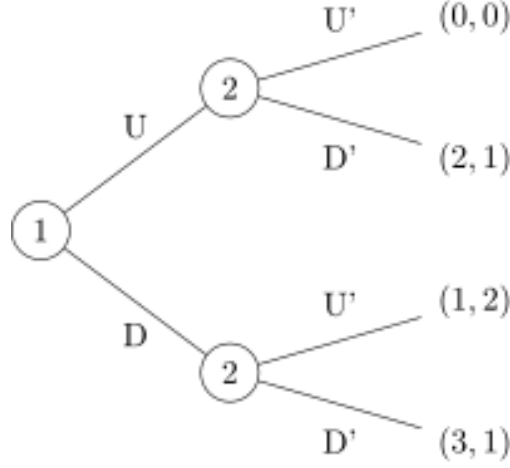


Figure 2.1: Simple example of a extensive game with prefect information. There are 2 players $P = 1, 2$ and each player can take action to go either up (U) or down (D). After both players have taken actions, the game reaches a terminal state and each player are given payoff. Notice this is not zero-sum, as the payoff does not add up to zero.

hence the other name tree-form game.

Extensive games can have either perfect of imperfect information. A perfect information game is a game where all players can observe the complete game state (or all information that is significant in calculating the optimal action). Chess, Shogi and Go fall into this category. In this work we focus on imperfect information games, as the subject of study, Pokemon, falls into this category.

In an imperfect-information extensive-form game there is a finite set of players P . A node, or *history* h , is defined by all information of the current game state, including private knowledge observed only by one of the two players. We assume the two players take turn to act one by one, $P(h)$ denotes the unique player who acts at node h , or chance if no player can act on that node(initial and terminal nodes). $A(h)$ denotes available actions to

$P(h)$ on that node. If an action $a \in A(h)$ leads h to h' , this is written as $h \cdot a = h'$. If there exists a sequence of actions that leads from h to h' , such fact is denoted as $h \sqsubset h'$. $z \in Z$ is a terminal node where the game trajectory concludes and no action is available. For each player $p \in P$, there is a payoff function $u_p : Z \rightarrow \mathbb{R}$ that returns a scalar payoff(reward) for that player. As mentioned before, payoffs of all players sum up to zero in zero-sum games. The range of payoffs is denoted as *Delta*. In the context of Poker gambling, *Delta* would mean the maximum amount of money one can win or loss.

Imperfect information is represented by *information sets*(infosets) for each player $p \in P$. Every infoset $I = \{h_0, h_1, \dots\}$ belongs uniquely to one player. To that player, all nodes in I are indistinguishable. For example in Poker, an infoset might look like

```

1 Infoset1 = {
2     "myCards" = ["4", "Q", "K"],
3     "opponentCards" = ["?", "?"]
4 }
```

It describes the situation where the player has three cards: "4", "Q", and "K", and the opponent has two cards which the player do not know. If we expand this infoset into nodes, it will look like

```

1 print(Infoset1.expand())
2 # >> {
3 # {
4 # "myCards": ["4", "Q", "K"],
5 # "OpponentCards": ["1", "1"]
6 # },
7 # {
8 # "myCards": ["4", "Q", "K"],
```

```

9  # "OpponentCards": ["1", "2"]
10 # },
11 # {
12 # "myCards": ["4", "Q", "K"],
13 # "OpponentCards": ["1", "3"]
14 # },
15 # .....
16 # }

```

It simply contains all possibilities of the opponents two unknown cards, and collects all of those possible nodes into a set. Every non-terminal node $h \in H$ belongs to exactly one info set for each p , as the game tree of both Poker and Pokemon cannot cycle back to previous turns. The set of all info sets belonging to p is represented as $\mathcal{I}_p$. We call the set of all terminal nodes with a prefix node in I as Z_I , and the particular prefix node is referred to as $z[I]$.

A strategy (or policy) $\sigma(I)$ is a probability vector over $A(I)$ that characterize how the player acts on I . This is to be compared against policy (often denoted as π) in the context of reinforcement learning. A RL policy maps a observation to a discrete action (although stochastic policies also exist in RL), while the strategy here outputs a probability vector that represents how likely the player is to play each available actions. Probability of a particular action a is denoted as $\sigma(I, a)$. The strategy for p across all info sets is defined as σ_p . In practice this can be represented in a info set to probability vector table, or approximated by a neural network. A strategy profile $\sigma = \{\sigma_p, \sigma_{-p}\}$ is a tuple of strategies of each player. $u_p(\sigma)$ or the equivalent $u_p \sigma_p, \sigma_{-p}$ is the expected payoff for p under profile σ .

$\pi^\sigma(h)$ is called **reach** of h . It is the probability of h being reached under

the profile σ . It is defined as follows:

$$\pi^\sigma(h) = \prod_{h' \cdot a \sqsubseteq h} \sigma_{P(h')}(h', a)$$

To translate this equation into plain English, the reach of h is calculated by first finding a node-action pair (h', a) that precede h , then identify which player acts on h' using the $P(h')$ function, calculate the probability of that player playing the action a . Now calculate this action probability for all (h', a) pairs and return the product of all action probabilities.

$\pi_p^\sigma(h)$ denotes the contribution of p to the reach of h and $\pi_{-p}^\sigma(h)$ denotes the same of the other player and chance. We can also define reach and contribution similarly to infosets. The probability of infoset I being reached is dependent on both how the player acts and how the other player and chance behaves. When a player is determined to reach I and trying the best to do so by choosing actions that leads to I , now the probability is solely dependent on the opponent and chance, as the player is doing his theoretical best. Chance of I being reach now is determined by the contribution of the opponent and chance alone, therefore should be denoted as $\pi_{-p}^\sigma(I)$ and this probability can be calculated as follows

$$\pi_{-p}^\sigma(I) = \sum_{h \in I} \pi_{-p}^\sigma(h)$$

which is simply adding up the non-player contribution of all nodes in the infoset of interest.

Finally consider a specific terminal node z . Remember if at least one action sequence exists such that non-terminal state h ends up in z , we write $h \sqsubseteq z$. For this h and z , we define the probability of h ending up at z [YOU

SURE ABOUT THIS?] as:

$$\pi^\sigma(h \rightarrow z) = \prod_{h' \cdot a \sqsubseteq z, h' \not\sqsubseteq h} \sigma_{P(h')}(h', a)$$

A *best response* to σ_{-p} is a strategy of player p denoted as $BR(\sigma_{-p})$ such that the expected payback for p is maximized. Formally:

$$BR(\sigma_{-p}) = \operatorname{argmax}_{\sigma_p} u_p(\sigma_p, \sigma_{-p})$$

A *Nash equilibrium*[12] σ^* is a strategy profile where every player plays a best response:

$$\forall p, u_p(\sigma_p^*, \sigma_{-p}^*) = \max_{\sigma'_p} u_p(\sigma'_p, \sigma_{-p}^*)$$

The *exploitability* of a strategy $e(\sigma_p)$ in a two-player zero-sum game is a measure of how much worse σ_p is playing against its best response $BR(\sigma_p)$ compared to how a Nash equilibrium strategy σ_p^* does against $BR(\sigma_p^*)$ in terms of expected payoff. Formally:

$$e(\sigma_p) = u_p(\sigma_p^*, BR(\sigma_p^*)) - u_p(\sigma_p, BR(\sigma_p))$$

And as a measure of how far a strategy is from the Nash equilibrium strategy profile σ^* , we can calculate the *total exploitability* of a strategy profile σ by simply adding up the exploitability of each player $\sum_{p \in P} e(\sigma_p)$. Some literature uses *average exploitability* instead.

2.2 Regret Minimization Methods

Regret, regret-matching and regret minimization methods is a family of concepts and algorithms aimed to calculate, or approximate, the Nash equilibrium of Poker, and other large extensive games like Poker. Most literature around this method almost exclusively focuses on solving the game of Poker, much like the history of Monte-Carlo Tree Search and the game of Go. Regret minimization, despite technically being a machine learning algorithm with an iterative update target and an implicit target function of approaching the Nash equilibrium, is rarely referred to with terms such as "machine learning" or "reinforcement learning".

The first paper that proposed the regret matching algorithm by Sergiu Hart and Andreu Mas-Colell did not mention for once the word "learning" or "machine learning", but instead categorized the method as "a adaptive procedure"[13]. The 2000 paper was published in the *Economica*, one of the top journals in the field of economics, telling the root of this idea lies in economics and game theory, rather than computer science and artificial intelligence. Reinforcement learning, on the other hand, emerged from the field of control theory, dynamics programming and a little bit of psychology and neuroscience.

Regret minimization algorithms have a different explicit objective than reinforcement learning algorithms. Regret minimization methods, as the name suggests, aims to minimize the *regret* and by which to approximate the **Nash equilibrium strategy** σ^* . Reinforcement learning methods focus on maximizing the *reward* function by either learning the optimal *policy* π^* or approximate a accurate *value function* $Q(s, a)$. Despite having different roots of ideas and explicit objectives, both families of algorithms can be employed to train strong agents in extensive games.

Furthermore, many of the ideas used in regret minimization methods resemble those in reinforcement learning. The very concept of *regret* is very similar to the concept of *Q-value* in RL. The Q-value function as in the context of RL maps a state-action pair to a scalar value, $Q(s, a) : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{R}$, that indicates how "good" the action under the specific state is. The regret function in the context of regret minimization methods maps a tuple of info set and action to a scalar regret value, $R(I, a) : \mathcal{I} \times \mathcal{A} \rightarrow \mathbb{R}$, which is indicative of the same as the Q-value.

Recently we start to see the trend of both families of methods starting to take on more complex real-world like problems and starting to merge into one toolbox of building generalized intelligent agents, with works like DeepMind's *Player of Games*[14] and *Rebel* by Facebook AI[15].

2.2.1 Regret and Regret Matching

In this subsection we will go through the definition of regret and its implications. This work adopts the notations used in the 2007 counter-factual regret paper[16] instead of the original 2000 paper to keep consistent with previous sections.

Consider repeatedly playing an extensive game, say Rock-paper-scissor(RPS). The **average overall regret** for player i on the $T - th$ round is defined as follows:

$$R_i^T = \frac{1}{T} \max_{\sigma_i^*} \sum_{t=1}^T (u_i(\sigma_i^*, \sigma_{-i}^t) - u_i(\sigma^t))$$

Translating this equation to English, the regret value represent how much worse as in payoff is the actual outcome $u_i(\sigma^t)$ compared to the best case scenario $u_i(\sigma_i^*, \sigma_{-i}^t)$. Calculate the regret for every single round and average the values over T rounds, you get the average overall regret R_i^T .

Regret matching is a popular regret minimization(RM) algorithm due to its simplicity and lack of parameters. Other famous regret minimization algorithms include the weighted majority algorithm [17] and [18]. In this thesis we focus on regret matching only. RM updates a player's strategy over multiple rounds (iterations) of the game. The update rules are defined as follows:

$$\sigma_i^{T+1}(I, a) = \begin{cases} \frac{R_i^T(I, a)}{\sum_{a \in A(I)} R_i^T(I, a)} & \text{if } \sum_{a \in A(I)} R_i^T(I, a) > 0 \\ \frac{1}{|A(I)|} & \text{otherwise} \end{cases}$$

To unpack, a regret matching strategy simply chooses action with probabilities in proportion to the average overall regret value of that action, given that the regret of all action $a \in A(I)$ sum up to a positive number. If the regrets does not add up to positive, choose all action at uniform probability. The intuition behind this process is that in general, the higher the regret value, the "better" an action is, and hence the higher probability in proportion. Typically a regret table is used to track regret values for all info-set-action pairs, and the table is updated at the end of each iteration. Strategy is also updated as a result of the updated regret table.

We will use a simple Rock-paper-scissor(RPS) example to illustrate the regret matching algorithm. Notice that in this demonstration we select p_0 to be the regret minimizing agent and we assume the opponent p_1 knows p_0 's strategy and will always play the best response to maximally exploit p_0 . When training regret minimization agents in practice, self-play is often used instead of training against best response opponents. Our agent p_0 here starts with a sub-optimal strategy of $\sigma_{t=0} = \text{Rock} : 0\%, \text{Paper} : 0\%, \text{Scissors} : 100\%$. We also define our payoffs: $u = \{\text{win} : +1, \text{draw} : 0, \text{lose} : -1\}$.

RPS can be modified into a extensive game, and arguable is the simplest

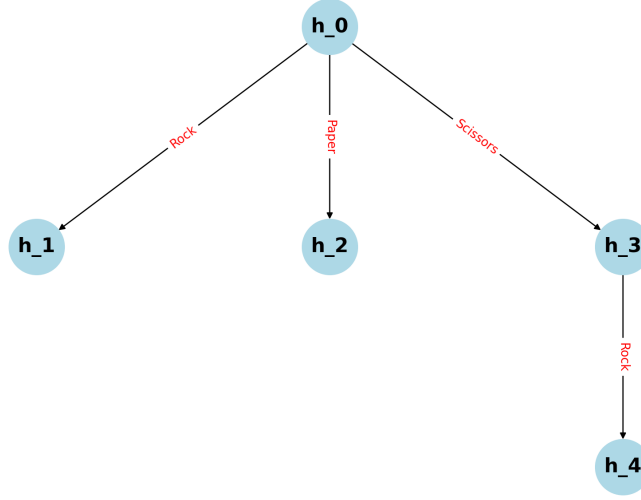


Figure 2.2: Simple example of an extensive game with perfect information. There are 2 players $P = 1, 2$ and each player can take action to go either up (U) or down (D). After both players have taken actions, the game reaches a terminal state and each player is given a payoff. Notice this is not zero-sum, as the payoff does not add up to zero.

one of the type. The RPS game in its extensive form can be represented as 2.2: starting from a beginning node h_0 where player p_0 takes one of three actions that leads to one of three subsequent nodes h_1 , h_2 or h_3 . And from each of the three, the other player p_1 will take an action which leads the game to a terminal state. Figure 2.2 illustrates one game sequence where p_0 played scissors and p_1 played rock, in which case p_0 loses the game, $u_{p_0} = -1$ and $u_{p_1} = 1$.

Now with a full trajectory of game play data $\mathcal{T}_0$ collected on iteration $t = 0$, we can start calculating regret values for each action we did not take and the overall regret on our strategy profile σ_{p_0} by aggregating those. Hold the opponent's action constant, which in this case is playing rock. If our agent played rock instead of scissor, the game would have been a draw, meaning the payoff for p_0 would have been $u_{rock} = 0$. Then the regret for not playing rock

is calculated by subtracting from the actual outcome from the hypothetical outcome where we have played rock, $R_{rock} = 0 - (-1) = 1$. Similarly, the regret for paper can also be calculated $R_{paper} = 1 - (-1) = 2$. The regret for scissors will be zero since we already played it. There is no regret in actions that have been already taken in reality. Note that in doing the regret calculation, the agent needs a simulator of the game to tell the hypothetical outcomes of which different actions were taken. This can be difficult to obtain if the game engine is unavailable and when the game engine does not exist in the first place. Recent works such as *Deep Regret Minimization with Advantage Baselines and Model-free learning(DREAM)*[19] dealt with this problem by using a deep neural network to approximate the behavior of the game. We will discuss DREAM in details in the following sections.

With the regret value calculated $R_0 = \{rock : 1, paper : 2, scissor : 0\}$, we can update our strategy to match the regrets: $\sigma_{t=1} = \{rock : 33.3\%, paper : 66.7\%, scissors : 0\%\}$. And play the next iteration with this updated strategy. Say this time we end up picking paper, which is action choice of the highest probability. Recall one of our initial assumptions that our opponent will always play the best response to our strategy. In this case, playing paper and scissors will have identical expected payoffs against our agent, so any combination of paper and scissors will do as a best response. Say the opponent also plays paper, then we have a draw on iteration $t = 2$. Regret value on this iteration will be calculated as: $R_{t=2} = \{rock : 0, paper : 0, scissor : 1\}$. Notice that the actual regret for rock this time is actually not zero but $R_{rock} = -1$, because if we played rock we would have lost the game, so we are happy we did not do it and hence the negative regret. In practice, ignoring negative regrets as zero do not change the convergence property of the algorithm and will speed up convergence in many cases, therefore it is common practice to

set negative regrets to zero $R^+ = \max(0, R)$. Then we have the final regrets for this iteration as $R_{t=1} = \{\text{rock} : 0, \text{paper} : 0, \text{scissors} : 1\}$.

Now we update our *average overall regret* table by taking the average of all iterations $t = 0, 1$, and conduct regret matching using the updated regret table. We continue this process for one more iteration, and the regret table and strategy over the first three iterations are shown in 2.1 and 2.2

iteration	R_{rock}	R_{paper}	R_{scissors}
0	1	2	0
1	0.5	1	0.5
2	1	0.67	0.67

Table 2.1: Evolution of regret values over the first three iterations

iteration	$\sigma(\text{rock})$	$\sigma(\text{paper})$	$\sigma(\text{scissors})$
0	0	0	1
1	0.33	0.67	0
2	0.25	0.5	0.25

Table 2.2: Evolution of strategy over the first three iterations of regret matching

We continue to run this update for 25,000 iterations. Figure 2.3 plots the action probabilities for rock, paper, scissors over the course. We can see that the strategy starts to converge to Nash equilibrium, which for RPS is simply equal probability for the three actions.

2.2.2 CounterFactual Regret(CFR)

Regret matching on *average overall regrets* works fine for simple games like RPS, but as the game trajectories and action space grow larger, traversing the entire game tree to calculate regrets on every single iteration soon becomes

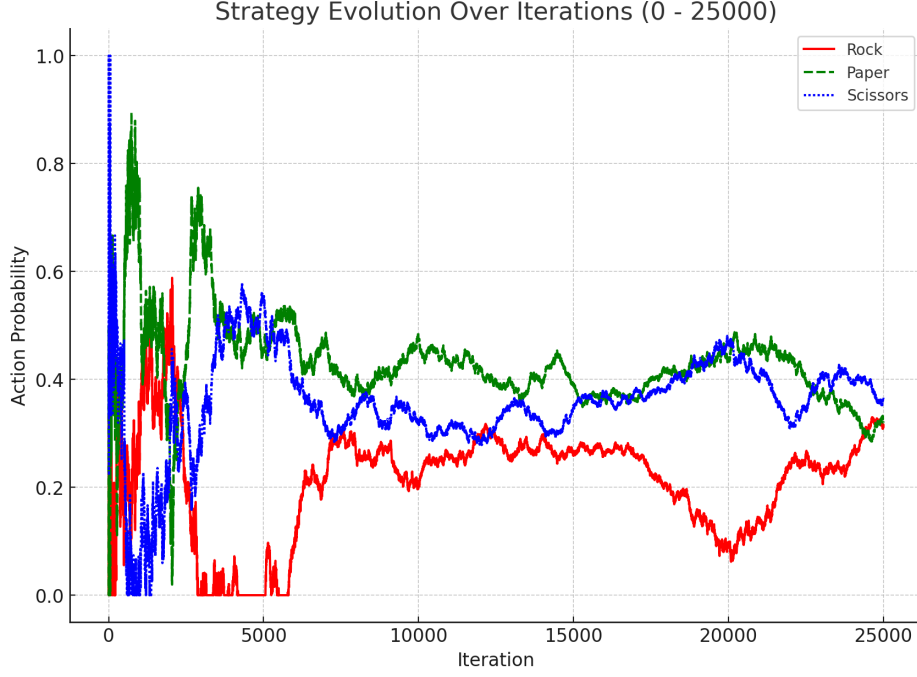


Figure 2.3: Example of applying self-play regret matching on rock-paper-scissors. After training for 25,000 iterations, action probabilities for all three actions start to converge to Nash equilibrium.

impractical. The idea of *counterfactual regret* is to decompose overall regret into a set of additive regret terms that can be minimized independently[16]. Here we go through the definition of counter-factual regret(CFR).

Consider a particular information set $I \in \mathcal{I}_i$, here subscript i denotes the particular player we are concerned about and $P(I) = i$. Define $u_i(\sigma, h)$ to be the expected utility (some literature use the notion *value* and *utility* interchangeably) given that the node h is reached and all players are playing under strategy profile σ . Similarly, define **counter-factual utility** $u_i(\sigma, I)$ to be the expected utility given that information set I is reached and all players play under σ except that player i plays to reach I . Formally, if $\pi^\sigma(h, h')$ is the probability of going from h to h' under σ , then

$$u_i(\sigma, I) = \frac{\sum_{h \in I, h' \in Z} \pi_i^\sigma(h) \pi^\sigma(h, h') u_i(h')}{\pi_{-i}^\sigma(I)}$$

To rephrase in plain English, the counterfactual utility of info set I simply means a weighted sum of utilities of all possible outcomes from node inside this info set. Closely related to counterfactual utility, there is also **counterfactual value**, which is defined as follows:

$$v^\sigma(I) = \sum_{z \in Z_I} \pi_p^\sigma(z[I]) \pi^\sigma(z[I] \rightarrow z) u_p(z)$$

The only difference between counterfactual utility $u_i(\sigma, I)$ and counterfactual value $v^\sigma(I)$ is that the latter is not scaled by $\frac{1}{\pi_{-i}^\sigma}$, and the latter is almost exclusive used in recent literatures [19]–[23]. It seems that the community has collectively decided that the scaling is unnecessary. Also a quick note on the difference in notation that in the former equation the particular player of concern is written as i while the latter uses p ; the former writes the current strategy profile σ as a argument to the utility function along with the input information set, while the latter places σ as a superscript of the value function. The two notation essentially mean the same thing: the final output of both function u, v depend on the current strategy profile σ .

And for all $a \in A(I)$, define $\sigma|_{I \rightarrow a}$ to be a strategy profile identical to I except that player i always chooses action a when in I . With that we can define the **immediate counterfactual regret** for player i on iteration T as:

$$R_{i,imm}^T(I) = \frac{1}{T} \max_{a \in A(I)} \sum_{t=1}^T \pi_{-i}^{\sigma^t}(I) (u_i(\sigma^t|_{I \rightarrow a}, I) - u_i(\sigma^t, I)) \quad (2.1)$$

To unpack, the last parenthesis term is the player's regret in its decisions

at info set I as measured in counterfactual utility. Notice that player i might have multiple available actions to take and different regret values for those actions at each iteration, but here we only take into account the particular action with the largest regret value for every iteration. This regret value is then weighted by the reach of I *if the player had tried to reach it*. And finally we take the average over all iterations to get the final immediate CFR. As stated in the previous section, we often will be only concerned about positive regret value, so we define the positive portion of immediate CFR as

$$R_{i,imm}^{T,+}(i) = \max(R_{i,imm}^T(i), 0)$$

One key property that makes CFR useful is that the overall regret is bounded by it:

$$R_I^T \leq \sum_{I \in \mathcal{I}} R_{i,imm}^{T,+}(I)$$

therefore minimizing immediate positive CFR on each iteration will guarantees to result in smaller overall regret, this allow us to approximate Nash equilibrium by minimizing immediate CFR alone. CFR can than be minimized using one of the regret minimization algorithms, such as regret matching.

Although CFR decomposes overall regret in the sense that now we can calculate CFR for each player on individual information sets, we still need to traverse entire game trees of the extensive game when calculating counterfactual value $u_i(\sigma, I)$ by plugging the possible terminal node into the utility function. This can be problematic when dealing with game with large game trees like Poker.

2.2.3 Monte Carlo CounterFactual Regret minimization(MCCFR)

Monte Carlo counterfactual regret minimization is a modification to the original CFR algorithm proposed in 2009[24]. It utilize Monte-Carlo sampling to approximate regret values instead of traversing complete game tree, making each regret update much faster than the original CFR update, while theoretically, and empirically, converging to the same ϵ -Nash equilibrium like the original. The paper also established a collection of notations that become common among succeeding studies.

As the core motivation of MCCFR is to trim the game tree, the first thing they did to achieve this is to limit the number of terminal nodes that are considered on each iteration. Let $\mathcal{Q} = Q_1, Q_2, \dots, Q_r$ be a set of subsets of all terminal nodes Z . Each of these subset, i.e. Q_1 is referred to as a **block**. The subscript r denotes the total number of blocks. On each iteration one of these block will be sampled and only terminal nodes inside that block will be considered for CFR calculation. Let $q_j > 0$ be the probability of block Q_j being considered for the current iteration, then naturally we would have $\sum_{j=1}^r q_j == 1$

Let $q(z) = \sum_{j: z \in Q_j} q_j$ be the probability of considering terminal node z on the current iteration. The **sampled counterfactual value** when updating block j is defined as follows:

$$\tilde{v}_i(\sigma, I|j) = \sum_{z \in Q_j \cap Z_I} \frac{1}{q(z)} u_i(z) \pi_{-i}^\sigma(z[I]) \pi^\sigma(z[I], z) \quad (2.2)$$

And therefore we can use the sampled counterfactual value to calculate the sample approximated immediate counterfactual regret by plugging equation 2.2 into equation 2.1. By dividing Z into smaller blocks, we make update on

each iteration faster. And suppose we choose $\mathcal{Q} = Z$, i.e. one block contains all possible terminal nodes and $q_1 = 1$, we have the vanilla CFR.

Sampled counterfactual value matches real counterfactual value on expectation (proof can be found in [24]), and therefore MCCFR can be used to approximate unbiased real Nash equilibrium.

There exist two sampling approaches when sampling block on each update, namely **Outcome Sampling**(OS) and **External Sampling**. We will describe how OS works in detail below since it is the one used in later deep learning-based methods.

Outcome-Sampling MCCFR

In *outcome-sampling* MCCFR we construct $\mathcal{Q}$ such that each block has exactly one terminal node, i.e., $\forall Q \in \mathcal{Q}, |Q| = 1$. On each iteration we sample one terminal node and only update information sets along that path. Since now blocks are equivalent to terminal nodes, the sampling probabilities q_j now must specify a distribution over all terminal nodes. We define this distribution to be a *sampling profile*, σ' , so that $q(z) = \pi^{\sigma'}(z)$. Clarifying on definition, σ' is a strategy profile that is defined on the collective action distributions of all players, just like all other sigmas are, and this sampling profile we just mentioned need to be configured such that under σ' , the reach of each terminal states $z \in Z$ forms the probability distribution that is $q(z)$.

We start the algorithm by sampling z using policy σ' , and store the probability of that particular being sampled, i.e., $\pi^{\sigma'}(z)$.

The sampled terminal history z is first *traversed forward* to compute each player's probability of playing to reach each prefix of the history, $\pi_i^\sigma(h)$. Then the history is *traversed backward* to compute each player's probability of playing the remaining actions of the history, $\pi_i^\sigma(h, z)$. During the backward

traversal, the sampled counterfactual regrets at each visited information sets are computed as follows:

$$\tilde{r}(I, a) = \begin{cases} w_I \cdot (1 - \sigma(a|z[I])) & , \text{if } (z[I]a) \sqsubseteq z \\ -w_I \cdot \sigma(a|z[I]) & , \text{otherwise} \end{cases}$$

$$\text{where } w_i = \frac{u_i(z)\pi_{-i}^\sigma\pi_i^\sigma(z[I]a, z)}{\pi^{\sigma'}()}$$

2.3 reinforcement learning and deep RL

The term "Reinforcement learning" (RL) refers to both the problem and solutions of training an autonomous agent to act in some dynamic environment in order to maximize some desired rewards. Think a new-born kitten living in the wild trying to stay alive for as long as she can. In order to survive, she needs to learn various sub-tasks such as hunting, avoiding higher class predators, staying warm from cold rains and so on. Each task requires her to have a valid understanding of the real external world and adjust her behaviours accordingly. Reinforcement learning, despite having its roots in psychology and biology, is about trying to achieve the same behaviour optimization through computational approaches for artificial agents.

The most commonly used formalization of agent-environment interactions in RL problems is Markov Decision Processes (MDPs) (Figure 2.4). In any time step t of the MDP, an agent observes a **reward signal** R_t and a either complete or incomplete **state representation** S_t from the **Environment** (agent's observed **states** are often also referred to as **observation** O_t , especially when the observed state does not fully characterize the environment). On receiving S_t and R_t , the agent uses some internal decision making mechanisms that maps S_t and R_t to an **action** A_t . The agent then executes the

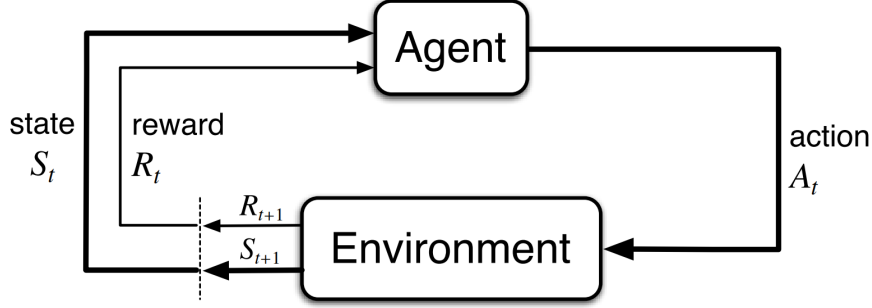


Figure 2.4: The agent-environment interaction is a markov decision process. Figure taken from [25]

selected action, usually having some impact on the environment, and then observes a new pair of S_{t+1} and R_{t+1} , and proceeds to the next time step. Over time steps the MDP will produce a *trajectory* τ

$$\tau = S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \dots \quad (2.3)$$

Trajectories are like labeled data in Supervised learning but it differs from labeled data in that, the trajectory are generated by learning agent itself and therefore depends on the current state of the agent, while supervised data usually has no dependency on the learning model.

The goal of RL agents is to learn the mapping from observed S to action A such that return G_t , the cumulative reward from now on (since the past cannot be changed), is maximized. The simplest way to define return is to sum all rewards from current time step to the end of episode:

$$G_t = R_t + R_{t+1} + R_{t+2} \dots + R_T \quad (2.4)$$

Here T is the termination time step of the episode. All kinds of problem,

however, arise for this simple sum definition of return. To name a few, its numerical value tend to be extremely unstable due to dependency on exponentially unpredictable future rewards and, for infinite MDPs, the return value could become infinite values too making it impossible to optimized on. A more practical and commonly used return is the sum of *discounted* rewards:

$$\begin{aligned} G_t &= R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{T-t} R_T \\ &= R_t + \gamma G_{t+1} \end{aligned} \tag{2.5}$$

Discounted rewards are used for all experiments in this work.

2.3.1 Reward Shaping

It is worth mentioning that the reward signal R in most RL systems need to be chosen/designed manually, and (if everything goes right) it is surprising how simple rewards can incentivize agents, especially those using deep neural networks, to learn extremely complex behaviours. If one wants to train a robot agent to move to designated goals as quick as possible, the reward can be set to -1 for every time step before it reaches the goal, such that maximizing the reward would be the equivalent of minimizing the time step the agent spends to reach the goal. To train an agent to play any 2-player zero-sum game such as chess or the game of GO, the reward can simply be set to 1 for winning the game and -1 for losing, such that maximizing reward would be the same as increase the chance of winning.

Simple rewards like +1 for winning might look like a beautiful solution on paper, but in reality agents often have hard time learning from such sparse rewards. Randomly initialized agents, like all beginners, usually perform very poorly on the task. If the agent only receives reward by winning and it never

wins, chances are it will never learn the correct actions that lead to victories. To avoid such failures, "training wheels", supplementary reward signals that are easier to access, can be given to the agents to initiate learning. The added rewards should be chosen to represent some sub-task that contributes to the main goal. Sticking with the chess example, in the early phase of training, the agent may be rewarded for taking out the opponent's pieces, such that it can learn to understand that taking out opponent's pieces is good. This technique of providing supplementary rewards is called **reward shaping** [26]. Once the agent has learned the sub-task, the reward shaping should be removed to prevent the agent from being trapped in local optimums.

2.3.2 Value functions, policies and models

All RL agents learn some policy π , with which the agents will pick its actions from. A policy $\pi(s)$ can be built on a value function $v(s)$. A value function is a function that maps states to a scalar "value", a score of how good that particular state is, according to the agent's current behaviour (policy). Once the agents know how good each action is, it can simply set its policy to be "pick the best action according to $v(s)$ ". Value functions are calculated and defined as the expected return starting from that state under some policy π . Value functions must be defined on some specific policy because different policies will result in different expected returns for the same states. Formally, value functions are defined as follows with the definition of return G_t introduced

in Equation 2.5:

$$\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] \\
&= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s\right] \\
&= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s']] \\
&= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \tag{2.6}
\end{aligned}$$

Equation 2.6 is the *Bellman equation* of $v_\pi(s)$, named after Richard E. Bellman. Here $\mathbb{E}_\pi$ denotes the expected value under policy π . The Bellman equation recursively decomposes the value of the current state $v(s_t)$ into the immediate reward r_t and the average of future state values $v(s')$ weight by the probability of occurrence. This notion of expressing the current in terms of future enabled by the Bellman equation will be essential throughout all RL algorithms.

$v_\pi(s)$ is often referred to as the *state-value function*, as it evaluates the value of state s . Another often used value function is the *action-value function* $q_\pi(s, a)$, also called the *q-function*, is defined as follows:

$$\begin{aligned}
q_\pi(s, a) &= \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \\
&= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a\right] \tag{2.7}
\end{aligned}$$

The q function evaluates the values of actions under specific states, following a given policy π . $v_\pi(s)$ can be expressed in $q_\pi(s, a)$ by simply adding q values for all possible actions, weighted by the probabilities of each action under policy π .

$$v_{\pi}(s) = \sum_a^{\pi(a|s)} \pi(a|s) q_{\pi}(s, a)$$

If the only possible action under some state s_t is a_t , then $v(s_t)$ would have the same value with $q(s_t, a_t)$, because the probability of the agent taking a_t will be 1. How agents can learn parametrized approximations of v and q are discussed in Section 2.5 and Section 2.6.

A policy can also be constructed without a value function. Such policies are usually direct mapping from state to actions. How such policies can be learned is discussed in section 2.7.

Optionally, agents can also have a internal representation of the outside environment, called model M . M can be either a perfect copy of the actual environment, or learned parametrized representation. Although the current landscape in RL research is mostly dominated by model-free agents, those who do not have a internal model, having a model can be handy in several situations. For example, game agents that relies on tree search algorithms, such as AlphaGo [6], need a internal model of the game to perform simulated lookaheads. The performance of AlphaGo increases by magnitudes when assisted with tree search, compared to using the estimations learned by neural networks alone. Ha and Schmidhuber [27], inspired by the "mental model" hypothesis [28] from the domain of psychology, proposed a deep neural network-based world model that can generate high quality simulated trajectories of data. They showed in their 2018 paper that agents trained on real + simulated data out-performs those trained on real trajectories alone.

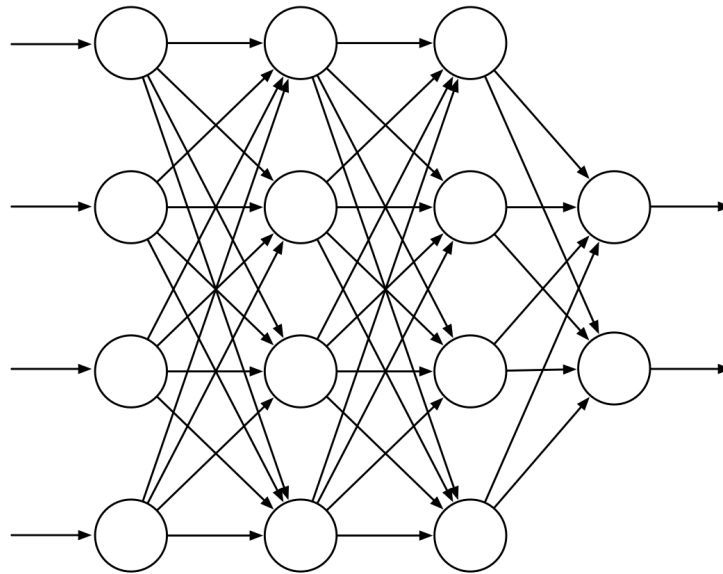


Figure 2.5: A generic feedforward Neural Network. Figure taken from [25]. Each circle represents a perceptron that calculated a weighted sum with bias for all its input and then activate the result with some activation function before sending it as output. Arrows represent inputs and outputs. For one perceptron, inward-pointing arrows are inputs coming from previous layers and all outward-pointing arrows represent the same output value of that perceptron.

2.4 Neural Networks

It has been almost a decade since Krizhevsky et al. crashed the ImageNet competition [5] with convolutional neural networks(CNNs) and neural networks has become more recognized than they were ever before. Various researches has shown that deep neural networks are capable of not only learning high-level representations on various types of data, but also use these learned features to perform various challenging tasks such as instance segmentation [29], machine translation[30] and so on.

Instead of trying to make decisions directly on the raw observed states, which are often complex and extremely high-dimensional, Deep RL agents can take the representations extracted by deep neural networks and then learn to approximate either a value function or a policy based on that representations, also using neural networks. All of these can be done in an end-to-end fashion and various researches has empirically proved that deep RL to be effective on complex tasks. To give an example, Deep Q-Networks(DQN) [31] is a deep RL algorithm proposed by DeepMind in 2013. The algorithm was demonstrated with an agent that plays atari games. The agent uses a 3-layer CNN to extract a 256-D vector representation from the original $84 \times 84 \times 4$ image input, then feed the vector representation into a fully-connected layers to calculate q-values for each action. DQN agents achieved state-of-the-art performance then and reached super-human performance in a few games such as Pong.

A neural network generally refers to a system, typically with a layered structure, that takes some numbers as input, applies a sequence of differentiable linear and non-linear computation to it and gives the result as output. A deep neural network refers to any neural networks with more than 1 hidden layer. Figure 2.5 shows a generic fully-connected neural network (FC

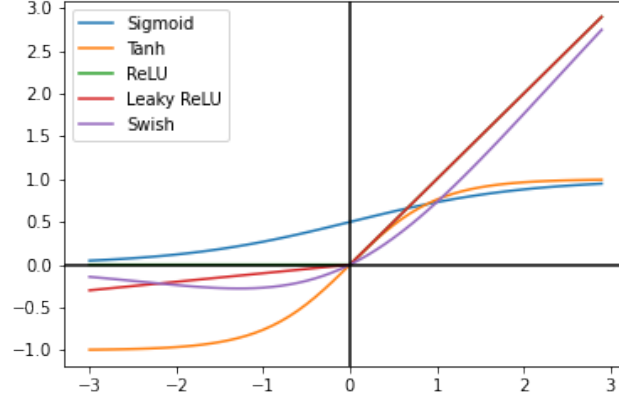


Figure 2.6: Commonly used activation functions in neural networks

net, also referred to as feed-forward neural network or multi-layer perceptron (MLP)) with one input layer with four neurons, two hidden layers both with four neurons and a output layer with two neurons. Each neuron stores a learnable weight vector $\mathbf{w}$ and a learnable scalar bias b . When provided a vector input, the neuron calculates the dot product between its weight and then adds its bias to produce the output. Inputs are depicted as arrows in Figure 2.5. After all outputs are calculated in a layer, a non-linear function (often called activation functions or simply activations) is applied to the sums before they are fed to the next layer. Some commonly used activations functions are shown in Figure 2.6. So to summarized, the i -th layer of a feed-forward network can be expressed in a nice compact form:

$$\mathbf{y} = \text{layer}_i(\mathbf{x}) = \sigma(f(\mathbf{x}; \theta_i)) \quad (2.8)$$

Where

$$f(\mathbf{x}; \theta_i) = f(\mathbf{x}; \mathbf{w}_i, \mathbf{b}_i) = \mathbf{x}^\top \mathbf{w}_i + \mathbf{b}_i \quad (2.9)$$

Here $\theta_i = \text{concatenate}(\mathbf{w}_i, \mathbf{b}_i)$ is the matrix holding all parameters in the

i-th layer.

2.5 Loss Functions

As mentioned in the previous subsection, the weights and biases in NNs are learnable. To make a NN "learn" means to optimized its parameters w.r.t some **objective function** using some data, typically through back-propagation and some gradient descend-like algorithm. Object functions are also often called "Loss functions" in the contexted of supervised learning.

Objective Function are hand-crafted functions that measures how well the NN currently is performing on the desired task. In supervised-learn, loss functions usually consist of some distance measure of the NN's output and the real label. The simplest loss functions for regression tasks is the L1 and L2 loss:

$$L1(\mathbf{x}, \theta) = |y - \hat{y}| = |y - f(\mathbf{x}; \theta)|$$

$$L2(\mathbf{x}, \theta) = (y - \hat{y})^2 = (y - f(\mathbf{x}; \theta))^2$$

Both measures the distance between the output(prediction) of NN using its current parameter θ and the correct output, only difference is in that L2 measures the square and L1 measures the absolute value. By minimizing these loss functions, the NN should learn to produce very similar outputs to the correct answers.

For classification tasks where the neural network is supposed to output a probability distribution of classes, the cross-entropy loss is a good starting point as a choice of loss function. For a k-class classification problem, the cross entropy loss is given as follows:

$$CrossEntropyLoss = - \sum_i^k t_i \log(s_i)$$

Here $\mathbf{t}$ is a k -dimensional one-hot vector containing the target label. For the input of i -th class, the label t should have $t_i = 1$ and all other entries as 0s. s_i is the NN's guess of the probability for input $\mathbf{x}$ being the i -th out of k total classes after a softmax activation. Formally:

$$\mathbf{s} = \sigma(f(\mathbf{x}; \theta))$$

$\sigma(\cdot)$ here is the softmax activation function:

$$\sigma(\mathbf{z}_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad (2.10)$$

Here $\mathbf{z}$ is the output of the last layer of the neural network f_θ . What the softmax activation does is applying an exponential to each entry of the vector to enlarge the difference between big and small values (softly selecting the biggest value, therefore called softmax), and then scales the numbers to make them sum to 1. By minimizing the cross-entropy loss, the NN will learn to give the highest probability (closest to 1) for the correct class of a given input and lower for wrong ones. Besides classification tasks, softmax activation is also frequently used in policy-based RL agents.

Reinforcement learning, unlike supervised learning, does not have so-called "target labels" or "correct answers", so the objective functions cannot be constructed by measuring the distance between NN's current prediction and some known desired outcome. The idea of optimizing parameter w.r.t some function, however, remains the same. Depending on what objective function is used, Deep RL algorithms are split into two families: value-based

algorithms and policy gradient-based algorithms. Value-base algorithms often use loss functions built on top of temporal-difference error (TD-error), defined in the following equation:

$$\delta_t = (R_{t+1} + \gamma V(S_{t+1})) - V(S_t) \quad (2.11)$$

Learning algorithms that use the TD-error are called TD-learning. TD-learning, like many other RL algorithms, has its origin in neurobiology and truly is a piece of genius. Here $V(S_t)$ and $V(S_{t+1})$ is the agent's estimated value of the current state S_t and the next state S_{t+1} . R_{t+1} is the immediate reward the agent receive on the current time step t . According to Equation 2.6, the value of the current state $v(s_t)$ should be equal to the immediate reward r_t plus the discounted value of the next state $v(s_{t+1})$, thus δ_t should always be zeros if the model's estimation $V(S)$ is perfectly accurate to the real value function $v(S)$. This, however, is usually not the case, especially in the early phase of learning, thus the TD-error will most likely has non-zero value.

Unlike supervised learning that fits the model's prediction to some correct outputs, TD-learning fits the agent's estimated value function $V(S_t)$ to the TD-target $r_t + V(S_{t+1})$. Notice the TD-target is not the same as target labels in supervised learning because it also uses the agent's estimation V . Fitting to the TD-target, however, still increases the accuracy of V iteration by iteration and eventually the learned $V(S)$ will fit to the real value function $v(S)$. This is because compared to the pure estimation $V(S_t)$, the TD-target is a mixture of estimation and a piece of information sampled from the real environment, which is the immediate reward r_t . This makes the TD-target a slightly more accurate representation of the real environment than the current estimation. And as the estimation gets better, the TD-target will also benefit from its estimation part and will always be slightly better than

the pure estimation. For this reason, the TD-error would work as a valid loss function for the agent to learn the true value function.

A more practical variation of TD-learning is Q-learning [32]. Q-learning use the same idea of minimizing TD-error, but learns the Q-function $Q(S, A)$ instead of the state value function $V(S)$. The TD-error for Q-learning is defined as follows:

$$\delta_t = (R_{t+1} + \gamma \max_a Q(S_{t+1}, a)) - Q(S_t, a_t) \quad (2.12)$$

Q-learning itself has a huge family of derived algorithms, famous ones includes the Deep Q-Networks, which use Q-learning to learn Q-function parametrized by deep neural networks.

To summarize, objective functions are manually designed functions that express the desired task which the designer of the NN wish it to learn. If the learning of the NN is conducted through gradient-based algorithms (which is most often the case), the objective function needs to be differentiable in terms of the NN's parameters.

2.6 Stochastic Gradient Descent and Back-propagation

Now we have a objective function $L(\theta; X)$ defined in terms of both the NN's parameters θ and the input distribution X . Assuming the input distribution is not changing, we can start optimizing the parameters using data. Or in other words, learning. Learning of NNs these days are almost all done with gradient-based algorithms. Once the gradient of the objective function w.r.t to the parameters $\nabla_{\theta} L(\theta)$ is calculated, the parameters can simply be

updated in that direction:

$$\theta^{t+1} = \theta^t + \alpha \nabla_{\theta} L(\theta; X) \quad (2.13)$$

Here t mean the number of iterations that the parameters have been updated. In reality, a dataset $\mathcal{D}$ (also called a batch) obtained from the true input distribution will be used to estimate the true gradient. This approach of using the whole dataset to compute a single step of gradient is called *gradient descent* [33], first proposed by Cauchy in 1847. Going over the whole dataset on every single step, however, can be costly in computation and could result in θ to be trapped in local optima. A more practical approach is *stochastic gradient descent* (SGD), which randomly devides the whole dataset into a bunch of minibatches and only use one minibatch to calculate gradient on each step. An epoch of SGD is illustrated in algorithm 1. Here epoch means a complete scan over the whole dataset $\mathcal{D}$. SGD is computationally much more efficient compared to gradient descent and empirically works much better on large dataset and networks. SGD and variants that utilized momentum such as *Adam* (Adaptive Moment Estimation) has been widely adopted for the superior empirical performance and will be used in this work.

Algorithm 1: One epoch of SGD

Given dataset $\mathcal{D}$;
Given current parameters θ_t ;
Given learning rate α ;
Devide dataset $\mathcal{D}$ into n minibachtes $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{n-1}$;
 $i \leftarrow 0$;
while $i < n$ **do**
 | **Backprop** using $\mathbf{x}_i$ to obtain estimated gradient $\hat{g}$;
 | **Update** $\theta_{t+1} \leftarrow \theta_t + \alpha \hat{g}$;
 | $i \leftarrow i + 1$;
end

To actually calculate the gradients of objective w.r.t to each single layer of the network, *Backpropagation* (backprop)[34] is used. Backprop is nothing more than a application of the chain rule of calculus. It calculates the gradient of the objective function starting from the last layer of the networks, then iteratively proceeds backwards. Since neural networks are differentiable by definition, backprop can go through all of the layers without problem, and end up with a gradient matrix $\nabla_{\theta}L(\theta; \mathbf{x}_i)$ that has the same shape of θ , ready to be used to perform the parameter update with the algorithm of choice, e.g., SGD.

2.7 Policy Gradient Methods

Besides learning a value function and building policies on the value function, RL agents can also learn an explicit representation of a direct mapping from state to action $\pi(s) \rightarrow a$, often parametrized by parameters θ . The objective function $J(\pi_{\theta})$ for policy optimization can be defined in several different ways, all of which should roughly express "How much reward the agent will get if it follows the policy π_{θ} ". And to update the policy, the agent needs to know the gradient of $J(\pi_{\theta})$ w.r.t. to the current policy parameter. This gradient is known as the *Policy Gradient*(PG) $\nabla_{\theta}J$. Once the PG is obtained, then the agent can simply update θ in the direction of $\nabla_{\theta}J$ and the performance will guarantee to improve.

The policy gradient, however, turns out to be tricky to calculate directly. Performance J is dependant on both the action distribution and the state distribution, and it is the latter that causes problems. Effect of policy parameter θ on the action selection is quite straight forward and can be calculated internally within the agent. Effect of θ on what states the agent will end

up in, on the other hand, usually also depends on the external environment and is not visible to the agents, and therefore cannot be calculated internally within the agent.

To avoid this problem we need to use the policy gradient theorem:

$$\nabla J(\theta) \propto \sum_s \mu_\theta(s) \sum_a q_\pi(s, a) \nabla \pi_\theta(a|s) \quad (2.14)$$

Here $\propto$ denotes "proportion to" and μ denotes the on-policy state distribution under policy π . The policy gradient theorem expresses the gradient of performance with terms with only the gradient information of the policy $\nabla \pi_\theta$, removing environment dependent terms in the differentiation process. The differentiation of policy can be simply estimated with samples from collected trajectories, which will be discussed later.

Learning policies directly offers a bunch of advantages over value-based methods. For example, sometimes it is much easier to just remember "turn left and this location" compared to "turning left worth 1456.7, turning right worth 1234.21, going straight worth 1400.42, doing nothing worth 0". This is just a random example, the point is that for the same task sometimes the policy can be significantly easier to learn compared to the value functions.

Another key reason to use PG methods is that it can learn stochastic policies. In some environments the optimal strategy is to play randomly. For example, in the game rock-paper-scissors, the optimal strategy is to play all 3 actions at a equal chance of $\frac{1}{3}$ because any strategy that favors any specific action can easily be exploited by the opponent and leads to suboptimal win rate that is lower than 50%. Value-based methods cannot learn stochastic policies because they have to take a deterministic max at test time.

2.7.1 REINFORCE

Now let's look at a classic PG algorithm which later state-of-the-art will be built on, the *REINFORCE* algorithm [35], also known as the "Vanilla Policy Gradient Algorithm". The idea of REINFORCE is to use Monte-Carlo sampled *experience*, tuples of (S, R, A) , to calculate estimated policy gradients and then perform update with SGD.

To derive the REINFORCE update, we start from the right-hand side of PG theorem, which expresses a sum over states $\sum_s \mu(s)$ weighted by how often the states occur if policy π is followed. We can rewrite this into an expectation form:

$$\begin{aligned}\nabla J(\theta) &\propto \sum_s \mu_\theta(s) \sum_a q_\pi(s, a) \nabla \pi_\theta(a|s) \\ &= \mathbb{E}_\pi \left[\sum_a q_{\pi_\theta}(S_t, a) \nabla \pi_\theta(a|S_t) \right]\end{aligned}\tag{2.15}$$

This form still requires a learned value function of the current policy q_{π_θ} , which we want to remove. So here is some math trick to remove it:

$$\begin{aligned}\nabla J(\theta) &= \mathbb{E}_\pi \left[\sum_a q_{\pi_\theta}(S_t, a) \nabla \pi_\theta(a|S_t) \right] \\ &= \mathbb{E}_\pi \left[\sum_a \pi_\theta(A_t|S_t) q_{\pi_\theta}(S_t, A_t) \frac{\nabla \pi_\theta(A_t|S_t)}{\pi_\theta(A_t|S_t)} \right] \\ &= \mathbb{E}_\pi \left[q_{\pi_\theta}(S_t, A_t) \frac{\nabla \pi_\theta(A_t|S_t)}{\pi_\theta(A_t|S_t)} \right] \\ &= \mathbb{E}_\pi \left[G_t \frac{\nabla \pi_\theta(A_t|S_t)}{\pi_\theta(A_t|S_t)} \right]\end{aligned}\tag{2.16}$$

Now the policy gradient can be calculated with information from only the current parametrized policy and sampled returns, yielding the REINFORCE

update:

$$\theta \leftarrow \theta + \alpha G_t \nabla \log \pi_\theta(A_t|S_t) \quad (2.17)$$

Here α is the learning rate and $\nabla \log \pi$ is just a compact equivalent to $\frac{\nabla \pi_\theta(A_t|S_t)}{\pi_\theta(A_t|S_t)}$. Note that this gradient is calculated for every single time step. A more practical way of performing parameter updates is to take the average gradient over a batch of steps and only performing one parameter update for each batch.

2.7.2 Baselines and Weighting Factors

The form of policy gradient giving in Equation 2.17 should theoretically converge to at least local optimums. Here G_t , or equivalently written as a function of state of a given time step $G(S_t)$, can be considered a dynamic "weighting factor" of how far the parameters should move in the direction of $\nabla \log \pi_\theta$ on each time step. The weighting factor that REINFORCE uses is in fact not a very good one, due to the unstable nature of the undiscounted return function $G(S_t)$. Fortunately, adding or subtracting any function $b(S_t)$ that only depends on the S_t from G does not change the expectation of the estimated PG, yielding a new valid PG update:

$$\theta \leftarrow \theta + \alpha (G(S_t) - b(S_t)) \nabla \log \pi_\theta(A_t|S_t) \quad (2.18)$$

This class of function $b(S_t)$ is referred to as the *baseline*. Various types of baselines has been explored to reduce variance historically. One specific baseline that has been historically successful is a learned state-value function $V_\phi(S_t)$ parametrized by ϕ . PG methods that involve learning a value function specifically **with bootstrapping** are called *Actor-Critic Methods*. Those that learns value functions without bootstrapping are not considered

to be Actor-Critic. For example, *Proximal Policy Optimization* (discussed below) learns a value function by simply minimizing mean square errors, and therefore is not considered to be part of the Actor-Critic family.

2.7.3 Proximal Policy Optimization(PPO)

Proximal Policy Optimization (PPO) [36], specifically PPO-Clip is the main RL algorithm used in this work and this section will discuss how it works.

A common problem with PG methods for learning parametrized policy is that the performance function is usually extremely bumpy, even with baselines. That is to say, small steps in the parameter space can cause complete performance collapse. PPO and its predecessor Trust Region Policy Optimization(TRPO) [37] are both designed to tackle this problem using the same idea to take a step as large as possible to maximize an objective function, but only doing so while satisfying some constraint that avoids such performance collapse. TRPO does so by limiting the KL-Divergence, a measure of how "different" two probability distributions are, between the new policy and current policy. TRPO empirically performs well, but sometimes can be computationally inefficient and is difficult to implement. PPO, specifically PPO-Clip puts constraints on parameter updates with a simple clipping mechanism without calculating any complicated divergence metrics. This make the algorithm much easier to implement while lowering the computation cost without sacrificing any performance.

The PPO parameter update is slightly different from how it is done in REINFORCE. Instead of taking steps in the direction of estimated policy gradient, each iteration of parameter update in PPO involves maximizing an

objective function:

$$\theta_{k+1} = \underset{\theta}{argmax} \mathbb{E}_{s,a \sim \pi_{\theta}} [L(s, a, \theta_k, \theta)] \quad (2.19)$$

Here k denotes the iteration of parameter updates and $L(s, a, \theta_k, \theta)$ is the PPO-CLIP objective function:

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), g(\epsilon, A^{\pi_{\theta_k}}(s, a)) \right) \quad (2.20)$$

Here $A^{\pi_{\theta}}$ is the advantage function under policy π_{θ} . Advantage is self-explanatory: it means how much better some action is compared to the average of all actions from a given state. Formally advantage is defined as:

$$A^{\pi_{\theta}}(s, a) = Q^{\pi_{\theta}}((s, a)) - V^{\pi_{\theta}}(s) \quad (2.21)$$

And sitting at the core of the PPO-Clip algorithm is $g(\epsilon, A)$, the clipping operation, which is defined as follows:

$$g(\epsilon, A) = \begin{cases} (1 + \epsilon)A & A \geq 0 \\ (1 - \epsilon)A & A < 0 \end{cases} \quad (2.22)$$

The intuition of the PPO-Clip objective is straight-forward. The agent would look at some real experience collected from the real environment, if an action a is good for a state s , which is to say if $A^{\pi_{\theta}}(s, a)$ is positive, the parameter should be updated in the direction that increases the chance of that action happening. If the action is bad, the opposite applies. But the step size of each iteration will be limited by a hyperparameter ϵ such that the new policy is not "too far" from the current policy.

2.8 Self-Play

All RL algorithm requires the agent to actually move in the environment. When the environment is a multi-player game, that would mean the agent would need to play against somebody in order to learn. The opponent that the agent trains against usually effects the end performance of the trained agent significantly. If the opponent is too strong, the learning agent will never win and therefore never receive any reward signal to learn from; if the opponent is too weak, the reward could be easily maximized with some trivial moves and the agent will never become stronger from there on. The process of designing appropriate task and opponents that matches the agents learning progress is called *curriculum learning*.

Curriculum learning is in itself an active area of study, but for RL, an neat solution to the curriculum problem already exists and is called *self-play*. Self-play is most commonly seen in 2-player games, where agents will be trained against a snapshot of itself from some point in the past (e.g. from the last update iteration). Self-play provides an elegant solution to the curriculum learning problem by using the learning agent itself as training opponent, the opponent will always at the right level of difficulty for the learning agent to learn to beat. The hope is that agent can start from random parameters, with zero human knowledge or intervention, learn to beat its past self iteration by iteration, and eventually achieve superhuman performance at any given game. First self-play research was done on checkers in 1959 [38] and since then various studies [39] [40] [7] have shown the effectiveness of self-play. The most famous one probably being [7], which reached superhuman performance in the game of Go using neural networks trained purely on self-play.

Building on top of self-play training, population-based training [40] (PBT) has been attracting attention in the past few years. Using the same idea of

using learning agents as training opponents, instead of using snapshots on the same learning agent, PBT trains a population of agents against each other while also combining ideas of evolution. PBT has achieved success in even more complex partially observable environments such as StarCraft [39]. Details of PBT will be further discussed in section 3.2.

2.9 Pokemon

Pokemon battle is a partially observable turn-based two-player game. Each player has a team of up to six Pokemons and 1-3 Pokemons will be active on the field for each player depending what rule set is being played. Each turn each active Pokemon can choose to make a move or switch to other Pokemon in the back each turn. Each Pokemon can learn one to four moves, and the effects of moves includes dealing damage to the opponent, applying negative status on the opponent, applying positive status on the user, manipulating the battle field and combinations of these effect. A game is won when all Pokemons of the opponents team is beaten.

2.9.1 Basic Damage Calculation

So the aim for the game in part can be interpreted as to do as much damage as possible to the opponent while taking as little as possible. To understand how the game is played one need to first understand how damage is calculated from a high level. The damage output, since generation 3, can be expressed as follows:

$$Damage = BasePower * Attack/Defence * STAB * Type * Others * Rand \quad (2.23)$$

The complete damage equation is rather verbose so here we decide to wrap half of the other terms into the *Other* term. Damage output first depends on the move's *BasePower*. Every move has a immutable base power, an integer value ranging from zero to 250. Damage also depends on the attacker Pokemon's *Attack* stats and the defender Pokemon's *Defence* stats. This stats of Pokemon depends on the Pokemon's species, nature and how they are trained by their trainer. *STAB* stand for "Same-type attack bonus", which values to 1.5 when a move is used by a Pokemon of the same type to the move, and 1 otherwise. *Type* refers to the Type effectiveness, which can take one value of 0, 0.25, 0.5, 1, 2 and 4. Type effectiveness will be further discussed in the next subsection. Others includes weathers, terrains, critical hit and other variables on which more advanced strategies can be built. Finally, a random number *Rand* from $0.85 - 1.0$ will be multiplied at the end of every damage calculation. Stochastic elements like this exists in every corner of the game, making Pokemon a game especially difficult for artificial agents to learn.

2.9.2 Type Effectiveness

One of the core fundamentals Pokemon players are expected to learn in the very beginning is the type chart, a chart. Both Pokemon and their moves has types. Moves have one type and Pokemons can have one or two types. Types have strength and weaknesses(refer to Figure 2.7), and damage output of the same move will double of halve depending on the defender's type.

Defender	Normal	Fire	Water	Grass	Electric	Ice	Fighting	Poison	Ground	Flying	Psychic	Bug	Rock	Ghost	Dragon	Dark	Steel	Fairy
Attacker	Normal	Fire	Water	Grass	Electric	Ice	Fighting	Poison	Ground	Flying	Psychic	Bug	Rock	Ghost	Dragon	Dark	Steel	Fairy
Normal													1/2	0			1/2	
Fire		1/2	1/2	2		2						2	1/2		1/2		2	
Water		2	1/2	1/2				2					2		1/2			
Grass		1/2	2	1/2			1/2	2	1/2		1/2	2		1/2	1/2		1/2	
Electric			2	1/2	1/2			0	2						1/2			
Ice		1/2	1/2	2		1/2		2	2						2		1/2	
Fighting	2					2		1/2	1/2	1/2	1/2	2	0			2	2	1/2
Poison				2				1/2	1/2			1/2	1/2				0	2
Ground		2		1/2	2			2		0		1/2	2				2	
Flying				2	1/2		2	2				2	1/2				1/2	
Psychic						2	2				1/2					0	1/2	
Bug		1/2		2			1/2	1/2	1/2	2				1/2		2	1/2	1/2
Rock		2				2	1/2	1/2	2		2						1/2	
Ghost	0									2				2		1/2		
Dragon															2		1/2	0
Dark							1/2			2				2		1/2		1/2
Steel		1/2	1/2		1/2	2						2					1/2	2
Fairy		1/2					2	1/2							2	2	1/2	

Figure 2.7: Type chart of Pokémon (Figure by Aussie Evil(CC0)). How to read this figure: to find the damage multiplier of Fire type on Grass type, first find Fire on the second row from the vertical Attacker column, then from that row find Grass type on the fourth column, which reads 2, therefore Fire type attacks have a 2X multiplier on Grass type Pokémon.

Chapter 3

Related Works

This chapter discuss recent state-of-the art publications in fields related to this thesis. Such fields includes deep reinforcement learning on games, deep regret minimization methods on games and previous research on solving the game of Pokemon.

3.1 AlphaGo

AlphaGo [6] and the later improved version AlphaZero [7] are arguably the most well-known AI project ever. The system that combined deep learning, Monte-Carlo tree search and self-play reinforcement learning has achieved unrepresented success, reaching superhuman performance in the ancient game of GO which of considered impossible by many before. It is hard to argue which one plays the most significant role among the three building pieces, AlphaGo is the first self-play reinforcement learning agent to reach superhuman performance in such sophisticated games, showcasing the potential of the training paradigm.

3.2 Population-based Training

Population-based training(PBT) [41] is a asynchronous optimization algorithm proposed by DeepMind in 2017. PBT describes a repeated schedule of training a population of agents initialized differently, removing the worse performing individuals and breed the next generation within the surviving elites with random mutation, essentially combining the advantages of learning and evolution. Experiments showed that PBT is effective in giving performance boost for not only reinforcement learning tasks such as playing video games, but also for supervised learning and unsupervised learning tasks such as machine translation and image generation with generative adversarial networks(GAN). Researchers from the same team later applied the algorithm to the video games of Quake III [40] and StarCraft [39] and achieved superhuman performance in both of them.

3.3 Self-play RL Pokemon Agent: Metagrok

Reinforcement learning on Pokemon has been studied in a 2019 study by Huang & Lee. [42] Their self-play RL-based agent "metagrok" trained on random teams achieve respectable performance, achieving a Glicko rating of 1677 on the official Pokemon Showdown server, which roughly approximate the skill level of amateur human players. They also attempted to fine-tune the agent trained on random teams to specific teams composed with a specific strategy of using trick room, a move that will reverse the relative speed of all Pokemon on the field, but the tuned agent turned out to perform worse and could not understand the intended strategy.

Chapter 4

Methodology

This chapter details the algorithms we implemented and tested, as well as experiment setup. We followed a modular principle in designing the experiments, that is, implement a baseline first, and then add possible improvements onto that baseline. To be specific, after setting up the environment, we first created a reinforcement learning baseline agent that is trained against a random action bot. Then we experiments with self-play variants and group self-play variants. We then implement a CFR agent trained against random bot and compared it with the reinforcement learning baseline. Results can be found in the next chapter: Results and Discussions.

4.1 Game engine and learning environment

We detail how we set up our environment for training in this section.

Our game engine of choice is Pokemon Showdown[43], an open-source implementation of the original proprietary Pokemon game in Node.JS and TypeScript. Showdown simulates the complete behavior of the original game, as well as extra features such as registering accounts, keeping track of past

game trajectories, team builders and more. Showdown hosts itself on a web server and other processes can establish real-time communication with the Showdown server using the websocket protocol. Our environment was setup using `node v14.21.1` and `Pokemon showdown` under the commit number of `ca0fe5d04`.

We use `Poke-env` to interface with Showdown. `Poke-env` is a open source project that implements the OpenAI gym interface for `Pokemon Showdown`. In providing the interface, `poke-env` also take care of the communication with the Showdown server using websockets, handling concurrent events and parsing the server response into nicely formatted Python objects. We used the version of `poke-env` under the commit number of `ca9925b`.

We use Python, PyTorch and TorchRL to implement our agents. All agents in the following experiments share the same architectures to control for variables. Each reinforcement learning agents would have two neural networks, the value network and the policy network within itself. (We also tested using a single double-headed actor-critic neural network where the input and hidden layers are shared between the value and policy head. This in theory should save on memory usage and reduce total parameters that need to be trained and therefore accelerate training. In practice, we did not see much difference compared to a two network setup. Having two separate networks also helps keep the agent architecture consistent with the later appearing CFR agents.) The value network is configured as a feed-forward neural network with 128-128 hidden neurons that takes in a 10-dimension vector as observation and maps to a value scalar. The policy network shares the same input and hidden dimension, but maps to a 22-D logit vector. The 22-D correspond to the action space of the game rule "gen8randombattle" which we train all our agents on. During training, these logits are used to

construct a categorical distribution for all possible actions, and actions are sampled from this distribution. At inference time, the agent simply takes the action with the highest probability (logit) as the action of choice.

For CFR agents: [TBD]

4.2 reinforcement learning baseline

We train our reinforcement learning agent against a random action agent on Showdown through poke-env. Each iteration of training consists of following steps.

Data collection - First, after everything is initialized, we start data collection by playing the game. On the end of each step, we collect a tuple of $\{s, a, r, d\}$, which are respectively the 10-D observation vector, the action that agent took, reward and whether the trajectory is finished. We keep collecting these tuples and store them into a *replay buffer*, under the buffer reaches its pre-configured size.

weights update - After we collected a batch of data, we randomly sample mini-batches from the replay buffer to perform stochastic gradient descent towards the objective using the Adam optimizer[44]. Our objective is three-fold: the PPO-Clip objective as detailed in previous chapters, the critic loss for fitting our value network on real values sampled from real data, and finally entropy to regulate exploitation and exploration[45]. We backpropagate all three objectives to update weight in the two neural networks. Our mini-batch size of choice is 512 and we perform 10 mini-batch update epoches on a single batch of data using sampling without replacement.

4.2.1 reward shaping

We used reward shaping when collecting the reward signals. In an ideal world, the reward should only come from winning the game, as it is the only thing that really matters for game agents playing a two-player zero-sum game. In reality, winning rewards appears extremely sparse as in only once per game for a game that on average lasts dozens, occasionally more than a hundred step. Extremely sparse reward signal will result in poor data efficiency as most data will simply not cause any meaningful update in weights and can make the learning process slow, highly stochastic and hard to reproduce. To counteract, we utilize *reward shaping*, by adding accessory reward signals based on heuristics. In our experiment, besides game wins and loses for +1 and -1 rewards, we add HP damage reward at $1\text{ HP} = 0.001$ and Pokemon defeat reward at $1\text{ Pokemon} = 0.1$ reward. Thus we ensure higher frequency of reward signals and implicitly bias the agent to do more damage and defeat more opponent Pokemons, while taking less damage itself and avoid getting it's own Pokemon defeated.

4.3 Self-play RL

To train a self-play agent, we start by initializing an RL agent like before with random weights, call it $p_{\text{learner}}^{\theta_0}$, where θ_0 is the randomly initialized parameters. Then we initialize the opponent agent $p_{\text{opp}}^{\theta_0}$ using the same parameters θ_0 . We then train p_{learner} in the same way as the baseline using the exact same hyperparameters. At the end of each update iteration, we evaluate $p_{\text{learner}}^{\theta'}$ against $p_{\text{opp}}^{\theta_0}$ for a hundred games, and record the win rate. Initially the win rate should be close to 50%, since both agent have similar weights. But as the learner agent learns, the win rate should gradually

increase. When the evaluation win rate exceeds 90% after T iterations, we overwrite the opponent agent’s weight with the latest learner weight θ . This will cause the evaluation win rate to drop back to around 50%. We continue training the learner against the upgraded opponent, until the maximum train step is reached. In our case, we trained for one million steps.

Self-play is an old idea that has been used since the 1960s. In reinforcement learning, self-play is connected to the idea of "curriculum learning" as explained in the 1993 paper by Jeffrey Elman[46]. Pokemon is a complex game and to win in this game is by no means a small task for randomly initialized neural networks. Agents needs to learn concepts of hundreds of pokemon species, the type chart, special v.s. physical moves and the list goes on. Self-play helps break the big goal of "learning to win" down into small feasible pieces. In the early stage of training, both the learner and the opponent would have little knowledge or skills to win the game. Therefore the learner only has to pick up one little thing, e.g. to abuse that one strong move that it learned to be effective, in order to win majority of the games. And then that little knowledge gets copied to the opponent, forcing the learner the pick up new skills, and eventually master the game by picking small skills one by one.

4.4 Population-based RL

Population-based training(PBT) as proposed in [40] is a training scheme based and expands on self-play. In our PBT experiment, we randomly initialize a pool of 30 agents $\mathcal{P}_i = p_0, p_1, \dots, p_{29}$ on different seeds, such that each agent will have different parameters at the beginning of training. We call this pool generation 0. Then we add in to the pool two strong heuris-

tic bots. Details on these bots will be discussed in the next chapter. We then randomly match agents and bots against each other. Each agent will collect their own replay buffer and perform weight updates on their own, using the data they have collected playing against various opponents. After 10 iterations, we stop training and start evaluating all 30 agents against a set of benchmark heuristic bots. We select the three top scoring agents, and overwrites weights of the remaining 27 agents with these 3 top performing agents' weight on equal proportion. With a new generation of agents $\mathcal{P}_\infty$, we continue this process until the max training step is reached.

PBT supposedly improves on self-play in two aspects. First it parallelizes parameter search, mitigating the risk of getting stuck in local optimums, while also making use of modern compute powers. It also takes in elements of evolutionary strategy in the sense that only the fittest survives to the next generation, giving PBT the potential to capture optimization paths that are hard to get on with gradient-based optimizers.

Chapter 5

Experiments and Results

In this chapter we first explain details of the heuristic benchmarks used for evaluating agent. We then present evaluation results of each experiment group detailed in the previous chapter. We discuss the difference between each agent groups and analyst causes of such differences.

5.1 Evaluation Opponents

Trained agents played against three heuristic bots and were evaluated on the win rate against each. The three evaluation bots are *RandomPlayer*, *MaxDamagePlayer* and *TypedMaxDamagePlayer*. *RandomPlayer* chooses random legal moves regardless of the state. *MaxDamagePlayer* chooses the moves with highest base power among current available moves. *TypeMaxDamagePlayer* chooses the move with the highest value of base power times type multiplier. Among the three evaluation targets, *RandomPlayer* is considered the weakest and *TypedMaxDamagePlaye* is considered the strongest. With the 8-dimension observation given to the smaller model, the best win rate it can have against *TypedMaxDamagePlaye* is 50%.

Table 5.1: Evaluation win rates against heuristic bots

Method	Random	MaxDamage	TypedMaxDamage
Train w/ heuristic player	0.74 $\pm$ 0.07	0.31 $\pm$ 0.11	0.14 $\pm$ 0.07
+ reward shaping	0.76 $\pm$ 0.05	0.32 $\pm$ 0.11	0.13 $\pm$ 0.08
Train w/ self-play	0.94 $\pm$ 0.01	0.66 $\pm$ 0.01	0.39 $\pm$ 0.02
+ reward shaping	0.95 $\pm$ 0.01	0.71 $\pm$ 0.02	0.45 $\pm$ 0.03
Train w/ competition	0.95 $\pm$ 0.01	0.70 $\pm$ 0.02	0.43 $\pm$ 0.03
+ reward shaping	0.95 $\pm$ 0.01	0.72 $\pm$ 0.02	0.46 $\pm$ 0.02

5.2 Experiment Results

The evaluation results are shown in Table 5.1. The table records the average and standard deviation of win rates of all 16 agents from each experiment group against three heuristic players for a thousand evaluation match. All agents record highest win rate against *RandomPlayer* and lowest against *TypedMaxDamagePlayer*, as expected. Adding reward shaping in most cases helps the agent to learn to become stronger, regardless of the training methods. With exact hyperparameters, observation and model, Agents trained with proposed methods appear to outperform those trained with self-play. Furthermore, reward shaping seems to have the smallest impact on agent performance in our proposed training method.

Looking at the results, we can tell that self-play agents relies more on having the guide of reward shaping to learn good performance, as reward shaping gives win rate boosts of 0.01, 0.05 and 0.06 to the agents evaluating against Random, Max and TypedMax respectively, compared to 0, 0.02 and 0.03 win rate boost observed in PBT agents. We also observed the behaviour of the trained agents from all groups. PBT agents appears to be most stable at selecting the correct moves on each turn while self-play agents sometimes

observe weird behavior, e.g. keep using setup moves such as *Calm Mind* for a few turns in a row, essentially wasting turns until the active Pokemon is defeated. This shows that, with the same training steps, PBT agents converged more consistently to the expected local optimum of parameters.

Chapter 6

Conclusions and future works

In this work we surveyed methods used to achieve super-human performance in the domain of go, chess and poker, and demonstrated these methods can also be effective in the much more complex game of Pokemon, even under very limited observation scope. This work compiles a comprehensive roadmap for future researchers who are interested in solving Pokemon and provides a handy manual to get them started.

Here we discuss a few directions for possible future works.

6.1 Expanding observation space

Our experiments limits agent observation to a 10-dimension vector due to resource limitation. In the actual Pokemon game, a player observes a tree-like structure of up to a hundred data points. Parsing these inputs can be challenging on its own. Skilled human players has the amazing ability to determine relevant subset of this enormous observation input with minimum effort. This process might be mimiced with some sort of the attention mechanism[47].

Stacking pass observations is also a technique used by systems like AlphaZero to achieve higher performance. This ideas might be applied to Pokemon too. By simply stacking 5 - 10 past observations, the agent would gain more insight of the ongoing dynamics from the input alone, mitigating the need to model key information such as the opponent's action distribution.

6.2 Importance of search

One consensus the computer Poker and computer Go research community seems to reach is the importance of search, specifically, Monte-Carlo tree search guided by neural networks. Search is the foundation of AlphaZero's superhuman performance as it is been shown in the ablation study that without search at inference time, AlphaZero's performance degrades significantly, falling way below the best human's elo rating[7]. Researchers from the poker community has also experimented converting Poker into a perfection information form to be compatible with search and have achieve some success in doing it[15], but the trade-off in the resulting astronomical action space and memory requirements are significant.

Bibliography

- [1] A. L. Samuel, “Some studies in machine learning using the game of checkers,” *IBM Journal of research and development*, vol. 3, no. 3, pp. 210–229, 1959.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hall, 2010.
- [3] G. Tesauro, “Td-gammon, a self-teaching backgammon program, achieves master-level play,” *Neural computation*, vol. 6, no. 2, pp. 215–219, 1994.
- [4] M. Campbell, A. J. Hoane Jr, and F.-h. Hsu, “Deep blue,” *Artificial intelligence*, vol. 134, no. 1-2, pp. 57–83, 2002.
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Advances in neural information processing systems*, vol. 25, pp. 1097–1105, 2012.
- [6] D. Silver, A. Huang, C. J. Maddison, *et al.*, “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, pp. 484–503, 2016. [Online]. Available: <http://www.nature.com/nature/journal/v529/n7587/full/nature16961.html>.
- [7] D. Silver, T. Hubert, J. Schrittwieser, *et al.*, “Mastering chess and shogi by self-play with a general reinforcement learning algorithm,”

- CoRR*, vol. abs/1712.01815, 2017. arXiv: 1712.01815. [Online]. Available: <http://arxiv.org/abs/1712.01815>.
- [8] J. Schrittwieser, I. Antonoglou, T. Hubert, *et al.*, “Mastering atari, go, chess and shogi by planning with a learned model,” *Nature*, vol. 588, no. 7839, pp. 604–609, 2020.
- [9] A. Mandhane, A. Zhernov, M. Rauh, *et al.*, “Muzero with self-competition for rate control in vp9 video compression,” *arXiv preprint arXiv:2202.06626*, 2022.
- [10] A. Fawzi, M. Balog, A. Huang, *et al.*, “Discovering faster matrix multiplication algorithms with reinforcement learning,” *Nature*, vol. 610, no. 7930, pp. 47–53, 2022.
- [11] A. Elluswamy. “Tesla ai day 2021,” Youtube. (2021), [Online]. Available: <https://www.youtube.com/live/j0z4FweCy4M?feature=share&t=4392>.
- [12] J. F. Nash Jr, “Equilibrium points in n-person games,” *Proceedings of the national academy of sciences*, vol. 36, no. 1, pp. 48–49, 1950.
- [13] S. Hart and A. Mas-Colell, “A simple adaptive procedure leading to correlated equilibrium,” *Econometrica*, vol. 68, no. 5, pp. 1127–1150, 2000.
- [14] M. Schmid, M. Moravcik, N. Burch, *et al.*, “Player of games,” *arXiv preprint arXiv:2112.03178*, 2021.
- [15] N. Brown, A. Bakhtin, A. Lerer, and Q. Gong, “Combining deep reinforcement learning and search for imperfect-information games,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 17 057–17 069, 2020.

- [16] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, “Regret minimization in games with incomplete information,” *Advances in neural information processing systems*, vol. 20, 2007.
- [17] N. Littlestone and M. K. Warmuth, “The weighted majority algorithm,” *Information and computation*, vol. 108, no. 2, pp. 212–261, 1994.
- [18] K. Chaudhuri, Y. Freund, and D. J. Hsu, “A parameter-free hedging algorithm,” *Advances in neural information processing systems*, vol. 22, 2009.
- [19] E. Steinberger, A. Lerer, and N. Brown, “Dream: Deep regret minimization with advantage baselines and model-free learning,” *arXiv preprint arXiv:2006.10410*, 2020.
- [20] N. Brown, A. Lerer, S. Gross, and T. Sandholm, “Deep counterfactual regret minimization,” in *International conference on machine learning*, PMLR, 2019, pp. 793–802.
- [21] M. Schmid, N. Burch, M. Lanctot, M. Moravcik, R. Kadlec, and M. Bowling, “Variance reduction in monte carlo counterfactual regret minimization (vr-mccfr) for extensive form games using baselines,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 2157–2164.
- [22] M. Johanson, N. Bard, M. Lanctot, R. G. Gibson, and M. Bowling, “Efficient nash equilibrium approximation through monte carlo counterfactual regret minimization.,” in *Aamas*, 2012, pp. 837–846.
- [23] O. Tammelin, “Solving large imperfect information games using cfr+,” *arXiv preprint arXiv:1407.5042*, 2014.

- [24] M. Lanctot, K. Waugh, M. Zinkevich, and M. Bowling, “Monte carlo sampling for regret minimization in extensive games,” *Advances in neural information processing systems*, vol. 22, 2009.
- [25] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, Second. The MIT Press, 2018. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>.
- [26] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Icml*, vol. 99, 1999, pp. 278–287.
- [27] D. Ha and J. Schmidhuber, “Recurrent world models facilitate policy evolution,” in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31, Curran Associates, Inc., 2018. [Online]. Available: <https://proceedings.neurips.cc/paper/2018/file/2de5d16682c3c35007e4e92982f1a2ba-Paper.pdf>.
- [28] J. W. Forrester, *World dynamics*. Wright-Allen Press, 1971.
- [29] X. Wang, R. Zhang, T. Kong, L. Li, and C. Shen, “Solov2: Dynamic and fast instance segmentation,” *Advances in Neural Information Processing Systems*, 2020.
- [30] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *NIPS*, 2017.
- [31] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, “Playing atari with deep reinforcement learning,” *CoRR*, vol. abs/1312.5602, 2013. arXiv: 1312.5602. [Online]. Available: <http://arxiv.org/abs/1312.5602>.
- [32] C. J. C. H. Watkins, “Learning from delayed rewards,” 1989.

- [33] A. Cauchy *et al.*, “Méthode générale pour la résolution des systemes d’équations simultanées,” *Comp. Rend. Sci. Paris*, vol. 25, no. 1847, pp. 536–538, 1847.
- [34] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [35] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Advances in neural information processing systems*, 2000, pp. 1057–1063.
- [36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal policy optimization algorithms*, 2017. arXiv: 1707.06347 [cs.LG].
- [37] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *International conference on machine learning*, PMLR, 2015, pp. 1889–1897.
- [38] A. L. Samuel, “Some studies in machine learning using the game of checkers,” *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, 1959. DOI: 10.1147/rd.33.0210.
- [39] O. Vinyals, I. Babuschkin, W. M. Czarnecki, *et al.*, “Grandmaster level in starcraft ii using multi-agent reinforcement learning,” *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [40] M. Jaderberg, W. Czarnecki, I. Dunning, L. Marris, G. Lever, A. Castaneda, *et al.*, “Human-level performance in first-person multiplayer games with population-based deep reinforcement learning. arxiv,” *arXiv preprint arXiv:1807.01281*, 2018.

- [41] M. Jaderberg, V. Dalibard, S. Osindero, *et al.*, “Population based training of neural networks,” *arXiv preprint arXiv:1711.09846*, 2017.
- [42] D. Huang and S. Lee, “A self-play policy optimization approach to battling pokémon,” in *2019 IEEE Conference on Games (CoG)*, 2019, pp. 1–4. DOI: 10.1109/CIG.2019.8848014.
- [43] smogon, *Pokemon-showdown*, Accessed on 2021-06-03, 2011. [Online]. Available: <https://github.com/smogon/pokemon-showdown> (visited on 06/03/2021).
- [44] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [45] V. Mnih, A. P. Badia, M. Mirza, *et al.*, “Asynchronous methods for deep reinforcement learning,” in *International conference on machine learning*, PMLR, 2016, pp. 1928–1937.
- [46] J. L. Elman, “Learning and development in neural networks: The importance of starting small,” *Cognition*, vol. 48, no. 1, pp. 71–99, 1993.
- [47] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.

APPENDIX A - Machine Specs

The specs of the computer used to experiments in this work are listed in the table below.

Table 6.1: Machine specs

Item	Value
CPU	Intel Xeon E5-2690
Memory	188G
OS	18.04.5 LTS (GNU/Linux 4.15.0-121-generic x86_64)

APPENDIX B - Derivation of the simplest form of policy gradient

Derivation of the simplest form of policy gradient is provided below.

$$\begin{aligned}\nabla_{\theta} J(\pi_{\theta}) &= \nabla_{\theta} \mathbb{E}_{\tau \sim \pi} [R(\tau)] \\ &= \nabla_{\theta} \int_{\tau} P(\tau|\theta) R(\tau) \\ &= \int_{\tau} \nabla_{\theta} P(\tau|\theta) R(\tau) \\ &= \int_{\tau} P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta) R(\tau) \\ &= \mathbb{E}_{\tau \sim \pi} [\nabla_{\theta} \log P(\tau|\theta) R(\tau)] \\ &= \mathbb{E}_{\tau \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R(\tau)]\end{aligned}$$

This is a expectation, which can be estimated with a sample mean. Denote the estimated policy gradient as $\hat{g}$:

$$\hat{g} = \frac{1}{D} \sum_{\tau \in D} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R(\tau)$$

APPENDIX B - Glossary

Table 6.2: Mathematical notations used in game definition

Symbol	Meaning
p	A player of the game
$\mathcal{P}$	The set of all players in the game
h	A node (history)
H	The set of all possible nodes in the game
$P(h)$	The unique player who acts at the node h , or chance
a	An action
$A(h)$	The set of all available actions at the node h
$h \cdot a = h'$	That the action $a \in A(h)$ leads from h to h'
$h \sqsubset h'$	There exist a sequence of action that leads from h to h'
z	A terminal node where no action is available.
Z	The set of all terminal nodes. $Z \subseteq H$
u_p	The payoff function for player p that maps $Z \rightarrow \mathbb{R}$.
Δ	Range of possible payoff $\mathbb{R}$.

Table 6.3: Mathematical notations used extensive games and regret minimization

Symbol	Meaning
$I = \{h, h', h'', \dots\}$	A information set(infoset).
$\mathcal{I}_p$	All infosets belonging to p by which p acts.
Z_I	All possible outcomes from I
$\sigma(I)$	A strategy(policy) unser I , a probability vector over $A(I)$.
$\sigma(I, a)$	The probability of $a \in A(I)$ being selected.
$\sigma = \sigma_1, \sigma_2$	A strategy profile
σ_p	A strategy for player p in every infoset where p acts.
σ_{-p}	The strategy of the player other than p
$u_p(\sigma_p, \sigma_{-p})$	Expected payoff for p under σ .
$\pi^\sigma(h)$	The <i>reach</i> of node h under σ .
$\pi_p^\sigma(h)$	Contribution of p to the reach of h .
$\pi_{-p}^\sigma(h)$	Contribution to π^σ of other players and chance.
$\pi_{-p}^\sigma(I)$	The chance for p to reach I on best effort.
σ^*	A <i>Nash equilibrium</i> strategy profile.
$e(\sigma_p)^*$	The <i>exploitability</i> of player p 's strategy
$v^\sigma(I)$	The <i>counter-factual value</i> of I to $p = P(I)$.
$r^t(I, a)$	The <i>instantaneous regret</i> of playing (I, a) on iteration t.
$R^T(I, a)$	The <i>counterfactual regret</i> for (I, a) at iteration T.

Table 6.4: Mathematical notations used in reinforcement learning

Symbol	Meaning
α	learning rate
γ	discount factor
S	state
A	action
R, r	cumulative and instantaneous reward
τ	a trajectory / an episode
G	return
t	a discrete time step
G_t	return at time step t
T	final time step of an episode
π	policy
π_θ	parametrized policy with parameter θ
$\pi(s)$	the action distribution given state s under policy π
$\pi(a s)$	probability of action a given state s under policy π
$\mathbb{E}$	expectation
$\mathbb{E}_\pi$	expectation under policy π
$v(s)$	state value of state S
$v_\pi(s)$	state value of state S under policy π
$q(s, a)$	action value of action a on state s
$q_\pi(s, a)$	action value of action a on state s under policy π
σ	activation function